

Performance and Energy Consumption in Mobile Applications

A Flutter Perspective

What is Mobile Performance?

1. Speed (Responsiveness)

The app's ability to react to user input and deliver content. This includes startup latency, smoothness of scrolling and animations, and the responsiveness of UI elements.

2. Efficiency (Resource Use)

The app's consumption of finite system resources. This includes memory (RAM) footprint, disk storage size, and, critically, its impact on battery life through energy consumption.

Core Metric: Startup Time

Start Type	System State	Process Status	Ideal Goal (Eng)	Android Vitals (Penalty)
Cold	App not in memory	Process created, runtime initialized	< 500ms	> 5s
Warm	Process in memory	Process reused, UI recreated	(Not specified)	> 2s
Hot	App & UI in memory	App brought to foreground	(Not specified)	> 1.5s

Core Metric: Rendering & Jank

The Frame Budget

The non-negotiable deadline to produce one frame. This deadline shrinks as device refresh rates increase.

- **60Hz:** $1000\text{ms} / 60 = 16.67\text{ms}$
- **90Hz:** $1000\text{ms} / 90 = 11.1\text{ms}$
- **120Hz:** $1000\text{ms} / 120 = 8.3\text{ms}$

Understanding Jank

Jank is the "visual hiccup" from missing the frame budget. It's caused by **Frame Time Variance**, not low average FPS.

A game with a **consistent 30 FPS** (33ms) feels smoother than an app with **55 FPS average** but spikes of 100ms+.

The Silent Drain

Module 2: Understanding Energy Consumption. Apps seen as "battery hogs" are uninstalled 2.5x more frequently.

Primary Energy Drivers



The Screen

OLED: Black pixels are "off" (0 power).
Dark Mode saves massive energy.

LCD: Backlight is always on. Dark Mode
provides no battery benefit.



CPU / GPU Load

Inefficient code causes high CPU load,
which generates heat. The OS throttles
the CPU to cool it, which then causes
jank. An energy problem becomes a speed
problem.



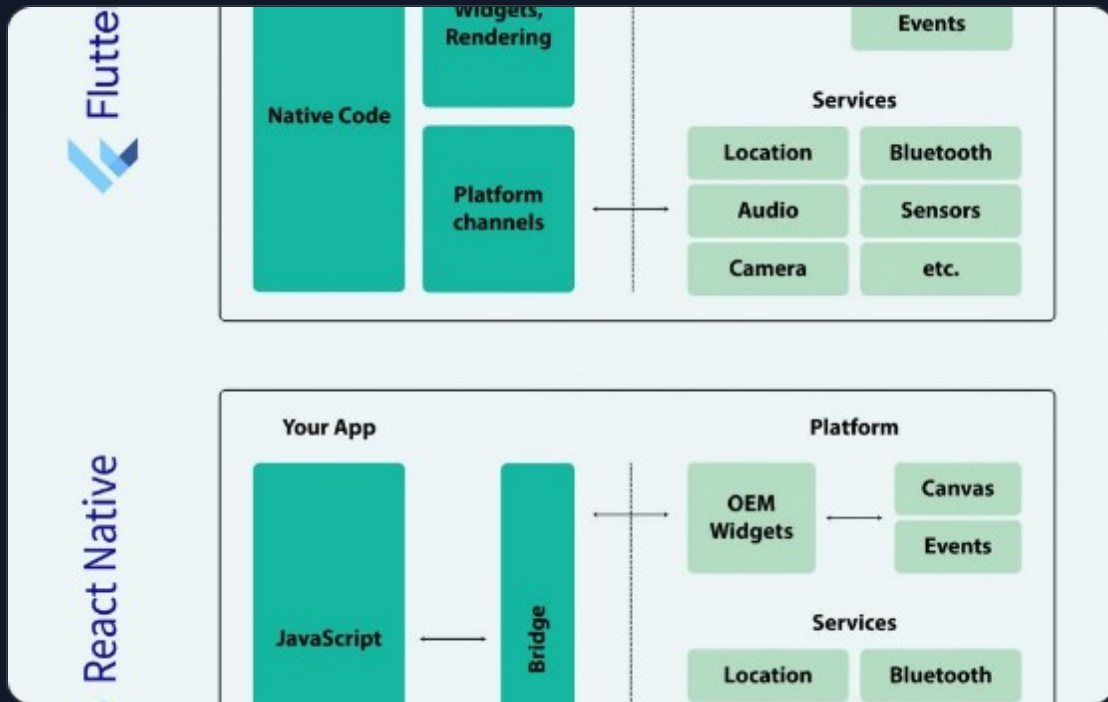
Network Radio

The "Energy Tail": The radio stays in high-
power mode for 5-10 seconds *after* a
request. "Chatty" apps with frequent small
packets never let the radio sleep.

Flutter's Architecture

Module 3: How Flutter Changes the
Equation.

Flutter "Owns the Pixels"



No Native Widgets

Flutter does not act as a "bridge" to native platform widgets (like React Native). It ships its own rendering engine (Skia/Impeller) and paints every single pixel directly to the screen.

Pro: Pixel-perfect UI consistency and AOT-compiled native speed.

Con: Flutter is 100% responsible for the performance of its own widget set.

Flutter's Rendering Pipeline & Engine

The Three Trees

Flutter's optimization strategy is to separate configuration from work:

- **1. Widget Tree:** The blueprint. Immutable and cheap to rebuild.
- **2. Element Tree:** The mediator. Holds state and compares widget trees.
- **3. Render Tree:** The worker. Does layout and painting. Very expensive to touch.

Engine: Skia vs. Impeller

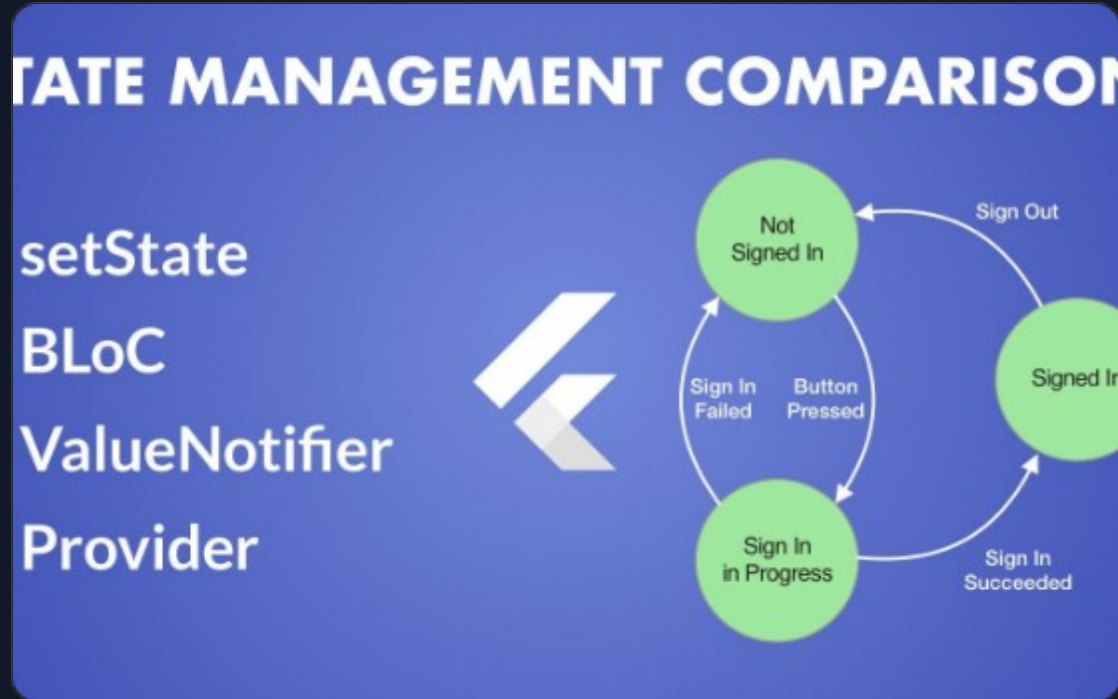
Skia (Old): Powerful, but suffered from **runtime shader compilation**. This was the cause of "first-run jank" (e.g., the first time a blur effect appeared).

Impeller (New): Built for Flutter. Has **no runtime shader compilation**. All shaders are pre-compiled at build time, guaranteeing predictable performance.

Bottlenecks & Fixes

Modules 4 & 5: Diagnosing and Fixing Common
Problems

Bottleneck: `setState()` & `const`



Stop Rebuilding Everything!

Problem: Calling `setState()` high in the widget tree (e.g., on the whole screen) forces Flutter to rebuild the *entire* UI for one tiny change.

Fix 1 (State): Use a state management solution (like Provider or Bloc) to rebuild *only* the specific widget that needs to change.

Fix 2 (Code): Use `const` constructors. This is the easiest win. It tells Flutter to "short-circuit" the rebuild, skipping the entire widget subtree.

Bottleneck: Lists & Heavy Work

`ListView` vs. `ListView.builder`

Bad: `ListView(children: [ ... 1000 items ... ])`

This builds ALL 1,000 items at once, even those off-screen. It causes a long app freeze and massive memory bloat.

Good: `ListView.builder()`

This **lazy loads** items, building them only as they are about to scroll onto the screen. This keeps CPU and memory usage low and constant.

Offload Work with `compute()`

Problem: Running heavy, synchronous work (like parsing a large JSON file or running a complex algorithm) on the main UI thread will **freeze the app**.

Fix: Use `compute()` (Isolates). This spawns a new isolate (like a thread, but no shared memory) to do the heavy work, keeping the main UI thread free to render frames 60x per second.

Bottleneck: Deceptively Expensive Widgets

Opacity and ClipRRect are not cheap. They are a primary cause of jank, especially in animations or ListViews.

 **They trigger a "saveLayer" operation.** This is an extremely GPU-intensive, multi-step process:

- 1** The engine must allocate a **new off-screen buffer** in GPU memory.
- 2** It must render the **entire child subtree** into this new buffer.
- 3** It then applies the opacity or clip to the buffer as a single texture.
- 4** Finally, it composites this buffer back onto the main screen.

 **This is a major bottleneck.** Use FadeInImage or other, more efficient methods when possible.

The Analyst's Toolkit: Flutter DevTools



How to Find the Bottleneck

1. **ALWAYS** run in Profile Mode (``flutter run --profile``). Debug mode performance is not representative.
2. Look at the Performance Chart. A red bar is a janky frame. The bar's color tells you why:

Blue Bar (UI Thread) = Your Dart code is slow.

(e.g., an inefficient ``build()`` method, a bad ``setState()`` call).

Green Bar (Raster Thread) = The GPU is slow.

(e.g., you used an ``Opacity`` or ``ClipRRect``, causing an expensive ``saveLayer()``).